INSTITUTO TECNOLÓGICO SUPERIOR TECSUP

DISEÑO Y DESARROLLO DE SOFTWARE

CURSO: DESARROLLO DE SOLUCIONES EN LA NUBE

PROYECTO:

LeGrandHotel – Sistema web de gestión de hospedaje

TUTORIAL: PRUEBAS DE CARGA CON K6 PASO A PASO

AREQUIPA, 2026

Docente:

VÍCTOR CALVO

Presentado por:

- Huacca Ccaso, Diego Armando
- Marcani Leyva, Joseph Fredy
- Pinto Garate, Leo Mathiu
- Quispe Encarnación, Rusell

Arequipa – Perú

2026

Índice

1. Antes de empezar 3
1.1 Qué vamos a probar y por qué 3
1.2 El recorrido completo 3
1.3 Qué necesitas 3
2. Paso 1: instalar k6 4
3. Paso 2: levantar la base de datos 4
4. Paso 3: crear las tablas y cargar datos de prueba 4
5. Paso 4: crear los usuarios de prueba 5
6. Paso 5: ajustar los límites de peticiones 5
7. Paso 6: levantar la API 6
8. Paso 7: comprobar que la API está sana 6
9. Paso 8: prueba de humo (smoke) 6
10. Paso 9: prueba del límite de login 8
11. Paso 10: pruebas de carga por escalones 9
12. Paso 11: leer los resultados 12
12.1 Qué salió en nuestras pruebas 13
13. Paso 12: limpiar y dejar todo como estaba 13
14. Si algo sale mal 13
15. Resumen de comandos 14

1. Antes de empezar

Este tutorial explica, de principio a fin, cómo hacer las pruebas de carga de LeGrandHotel con k6 en una computadora con Windows, y por qué se hace cada paso. Al terminar tendrás los resultados en la terminal y un reporte HTML con gráficos de cada prueba.

1.1 Qué vamos a probar y por qué

Una prueba de carga simula a muchas personas usando el sistema al mismo tiempo. Sirve para saber, antes de que pase con usuarios reales, si la API sigue respondiendo rápido y sin errores cuando hay mucho movimiento (por ejemplo, la recepción llena en temporada alta). k6 lo hace con «usuarios virtuales» (VU): programas que repiten un guion de peticiones como si fueran personas.

Prueba	Pregunta que responde	Guion
Humo (smoke)	¿La API, la base de datos y la caché funcionan?	`tests/k6/smoke.js`
Límite de login	¿La API bloquea a quien prueba contraseñas sin parar?	`tests/k6/rate-limit.js`
Escalones 5 → 50 VU	¿Cuánto tarda la API a medida que hay más usuarios a la vez?	`tests/k6/reservas.js`

1.2 El recorrido completo

Paso	Qué se hace	Por qué
1	Instalar k6	Es la herramienta que genera la carga
2	Levantar la base de datos	La API necesita sus datos para responder
3	Cargar datos de prueba	Con una base vacía las consultas son instantáneas y la prueba no dice nada real
4	Crear los usuarios de prueba	Los VU inician sesión como recepcionistas, igual que el personal real
5	Ajustar los límites de peticiones	Si no, la API bloquea a k6 con 429 y se mide el bloqueo, no la velocidad
6	Levantar la API	k6 necesita una dirección a la que mandar las peticiones
7	Comprobar que la API está sana	Si algo falla antes de empezar, la prueba no sirve
8–10	Correr las tres pruebas	Cada una responde una pregunta distinta
11	Leer los resultados	Decidir si la prueba pasó y qué hay que mejorar
12	Limpiar y dejar todo como estaba	Las pruebas crean reservas falsas y bajan la protección del login

1.3 Qué necesitas

- Windows 10 u 11 con el proyecto LeGrandHotel descargado (carpeta C:\Proyectos\LeGrandHotel).
- PHP 8.2 y Composer, con las dependencias del proyecto instaladas (composer install dentro de api).
- Docker Desktop, para la base de datos MySQL.

- Dos ventanas de terminal: una para la API (queda ocupada mientras está encendida) y otra para k6.

Nota: todos los comandos se escriben desde la carpeta del proyecto, salvo que el paso diga otra cosa. Cada paso trae la versión para PowerShell y para cmd (Símbolo del sistema), porque las variables se escriben distinto en cada una.

2. Paso 1: instalar k6

PowerShell
```
winget install k6 --source winget
```

Cierra la terminal y abre una nueva (para que Windows encuentre el comando k6). Luego:

PowerShell
```
k6 version
```

Por qué: k6 es un programa aparte del proyecto. winget es el instalador oficial de Windows: descarga k6 de Grafana y lo agrega al PATH, así el comando k6 funciona desde cualquier carpeta. Se abre una terminal nueva porque las ya abiertas no ven el PATH actualizado.

Qué debes ver: la versión instalada, por ejemplo «k6.exe v2.2.0». En cmd el comando es el mismo.



Figura 1. k6 instalado: el comando k6 version responde.

3. Paso 2: levantar la base de datos

Abre Docker Desktop y espera a que diga que está corriendo. Después, desde la carpeta del proyecto:

PowerShell
```
docker compose up -d mysql
```

Por qué: el proyecto usa MySQL 8 dentro de Docker, así todos tenemos la misma versión sin instalarla en Windows. Se levanta solo el servicio mysql (y no todo el docker compose) porque el contenedor nginx del proyecto usa el puerto 8000, el mismo que usaremos para la API; si los dos están encendidos, k6 habla con el equivocado y recibe 503.

Qué debes ver: «Container legrandhotel-mysql-1 Started». En cmd el comando es el mismo.

4. Paso 3: crear las tablas y cargar datos de prueba

PowerShell
```
cd api
php artisan migrate
php artisan db:seed --class=DemoSeeder
php artisan db:seed --class=MassiveDataSeeder
```

Por qué: migrate crea las tablas. DemoSeeder carga los roles, tipos de habitación, habitaciones e insumos de demostración. MassiveDataSeeder agrega 1000 huéspedes y 5000

reservas: una prueba de carga sobre una base casi vacía da tiempos irreales, porque buscar en 10 filas es instantáneo y en miles no.

Qué debes ver: cada seeder termina con «DONE». Si la base ya tenía estos datos (por ejemplo, ya lo hiciste antes), no vuelvas a correr MassiveDataSeeder: agregaría otras 5000 reservas.

5. Paso 4: crear los usuarios de prueba

PowerShell
```
$env:K6_USERS="50"
php artisan db:seed --class=LoadTestUsersSeeder
```

cmd
```
set "K6_USERS=50"
php artisan db:seed --class=LoadTestUsersSeeder
```

Por qué: cada usuario virtual inicia sesión como un recepcionista distinto (k6-0@carga.legrandhotel.test hasta k6-49@…), con su propio token, igual que el personal real. Si todos usaran la misma cuenta la prueba sería menos realista. Se crean 50 porque la prueba más grande usa 50 usuarios a la vez. Son cuentas solo para pruebas: nunca se crean en la base de producción.

Qué debes ver: «DONE». La variable K6_USERS le dice al seeder cuántas cuentas crear.

Ojo en cmd: escribe set "VARIABLE=valor" con las comillas así. Si escribes set VARIABLE=valor && otro-comando, cmd guarda un espacio al final del valor y la variable queda mal.

6. Paso 5: ajustar los límites de peticiones

Abre api\.env y deja estas tres líneas así:

api\.env
```
RATE_LIMIT_API=100000
RATE_LIMIT_LOGIN=100000
RATE_LIMIT_REGISTER=100000
```

Luego limpia la configuración guardada:

PowerShell
```
php artisan config:clear
```

Por qué: la API protege el login (5 intentos por minuto) y el resto de rutas (60 por minuto) contra abusos. k6 manda cientos de peticiones desde la misma computadora, así que con los límites normales la API lo bloquearía con 429 y estaríamos midiendo el bloqueo, no la velocidad. config:clear asegura que Laravel lea los valores nuevos del .env.

Importante: esto es solo para la prueba. En el paso 12 se vuelven a poner los valores normales.

7. Paso 6: levantar la API

En la primera terminal, dentro de la carpeta api:

PowerShell
```
php -S 127.0.0.1:8000 -t public public/index.php
```

Por qué: k6 necesita una dirección a la que mandar las peticiones. Se usa el servidor integrado de PHP en la dirección 127.0.0.1 (y no «localhost») porque en Windows localhost puede resolverse a la dirección IPv6 (::1) en un programa y a la IPv4 (127.0.0.1) en otro; si el servidor escucha en una y k6 llama a la otra, el 100 % de las peticiones fallan. Escribir 127.0.0.1 en los dos lados evita ese problema. Se usa php -S y no php artisan serve porque artisan serve no pasa las variables de entorno a la API, y en el paso 9 las necesitamos.

Qué debes ver: «Development Server (http://127.0.0.1:8000) started». Esta terminal queda ocupada: no la cierres mientras dure la prueba. En cmd el comando es el mismo.

Nota sobre las capturas: las capturas de este tutorial se tomaron con la API escuchando en la dirección IPv6 [::1]. Con 127.0.0.1 los comandos cambian solo en la dirección y la respuesta de k6 es la misma.

8. Paso 7: comprobar que la API está sana

En la segunda terminal, desde la carpeta del proyecto:

PowerShell
```
curl.exe -s http://127.0.0.1:8000/api/v1/health
```

Por qué: si la API, la base de datos o la caché fallan antes de empezar, la prueba de carga solo mostraría errores y no sabríamos por qué. /health responde en un segundo si todo está bien. En PowerShell se escribe curl.exe (con .exe) porque «curl» solo es un alias de otro comando.

Qué debes ver: un JSON con "status":"ok" en app, database y cache.

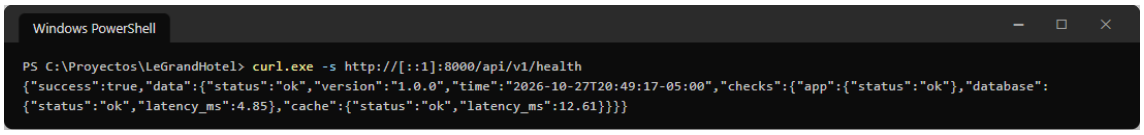


Figura 2. Respuesta de /health: la API está lista para la prueba.

9. Paso 8: prueba de humo (smoke)

PowerShell
```
k6 run -e BASE_URL=http://127.0.0.1:8000/api/v1 -e VUS=5 -e DURATION=30s
tests/k6/smoke.js
```

cmd
```
k6 run -e BASE_URL=http://127.0.0.1:8000/api/v1 -e VUS=5 -e DURATION=30s
tests/k6/smoke.js
```

Parte del comando	Qué significa
`k6 run`	Ejecuta un guion de k6

Parte del comando	Qué significa
`-e BASE_URL=...`	Le pasa al guion la dirección de la API (-e = variable para el guion)
`-e VUS=5`	5 usuarios virtuales a la vez
`-e DURATION=30s`	La prueba dura 30 segundos
`tests/k6/smoke.js`	El guion: pide /health y una ruta protegida sin token

Por qué: es la prueba más pequeña y va siempre primero: confirma que todo funciona con poca carga. Si esta falla, no tiene sentido seguir con las grandes, porque cualquier error vendría de algo roto y no de la carga. Además verifica que una ruta protegida sin sesión responda 401 en JSON (seguridad).

Qué debes ver: las tres metas con ✓ (THRESHOLDS) y los cuatro checks con ✓. Dura unos 30 segundos.

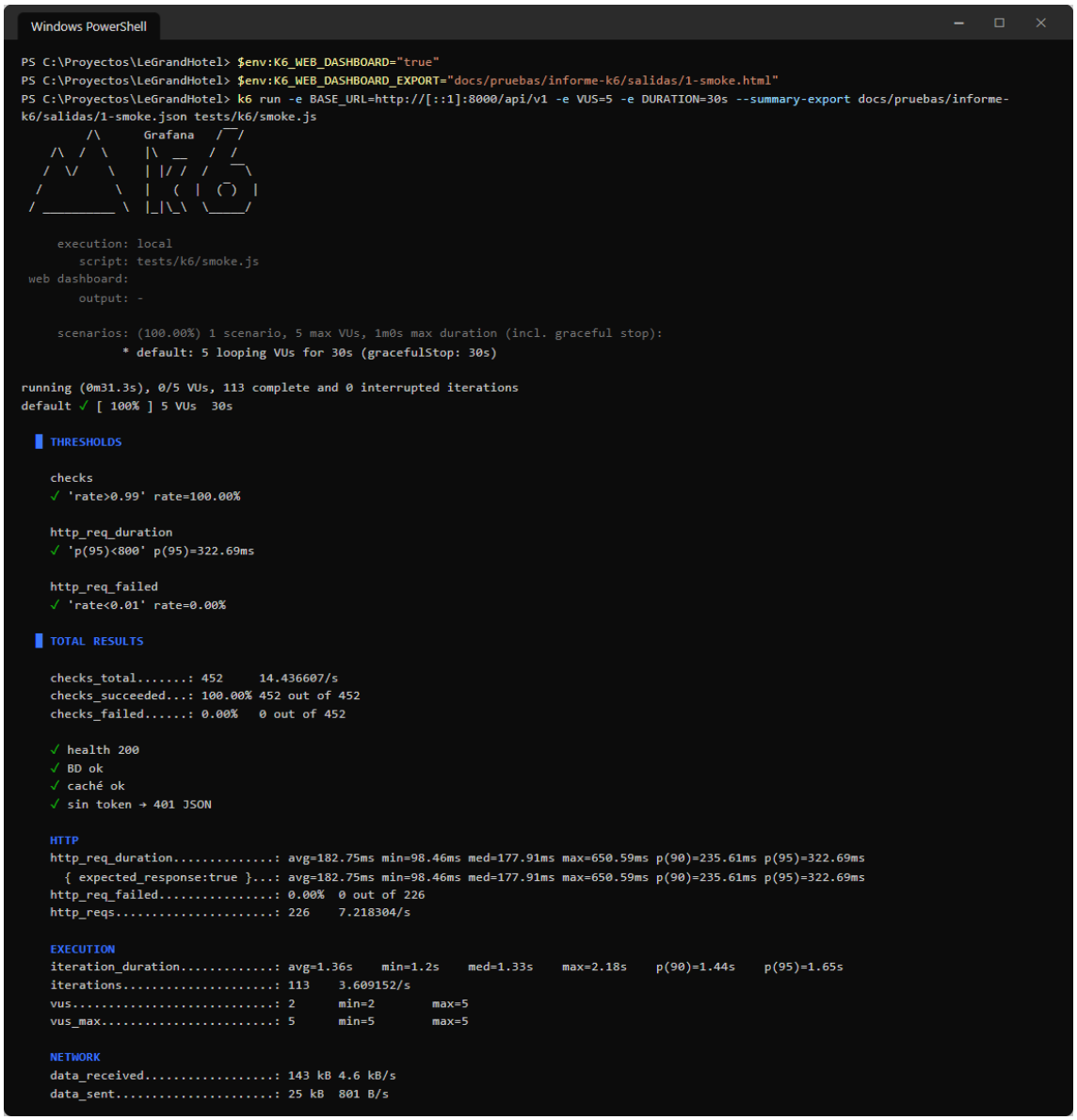


Figura 3. Prueba de humo aprobada: todas las metas y verificaciones con ✓.

10. Paso 9: prueba del límite de login

Esta prueba necesita el límite real de login. En una tercera terminal, dentro de la carpeta api, levanta una segunda API en el puerto 8001 con el límite de 5 intentos:

PowerShell

```
$env:RATE_LIMIT_LOGIN="5"
php -S 127.0.0.1:8001 -t public public/index.php
```

cmd

```
set "RATE_LIMIT_LOGIN=5"
php -S 127.0.0.1:8001 -t public public/index.php
```

Y en la terminal de k6:

PowerShell

```
k6 run -e BASE_URL=http://127.0.0.1:8001/api/v1 tests/k6/rate-limit.js
```

Por qué: en el paso 5 subimos los límites para las pruebas de velocidad, pero también hay que demostrar que la protección funciona. Se usa una segunda API con la variable RATE_LIMIT_LOGIN=5 (la variable de la terminal tiene prioridad sobre el .env), así no hay que editar el .env ni apagar la primera. El guion intenta entrar 6 veces con una contraseña incorrecta: los 5 primeros deben dar 401 y el sexto 429 «demasiados intentos».

Qué debes ver: «Estados recibidos: 401, 401, 401, 401, 401, 429» y los tres checks con ✓. Dura unos 3 segundos. Al terminar, cierra esta tercera terminal con Ctrl + C.

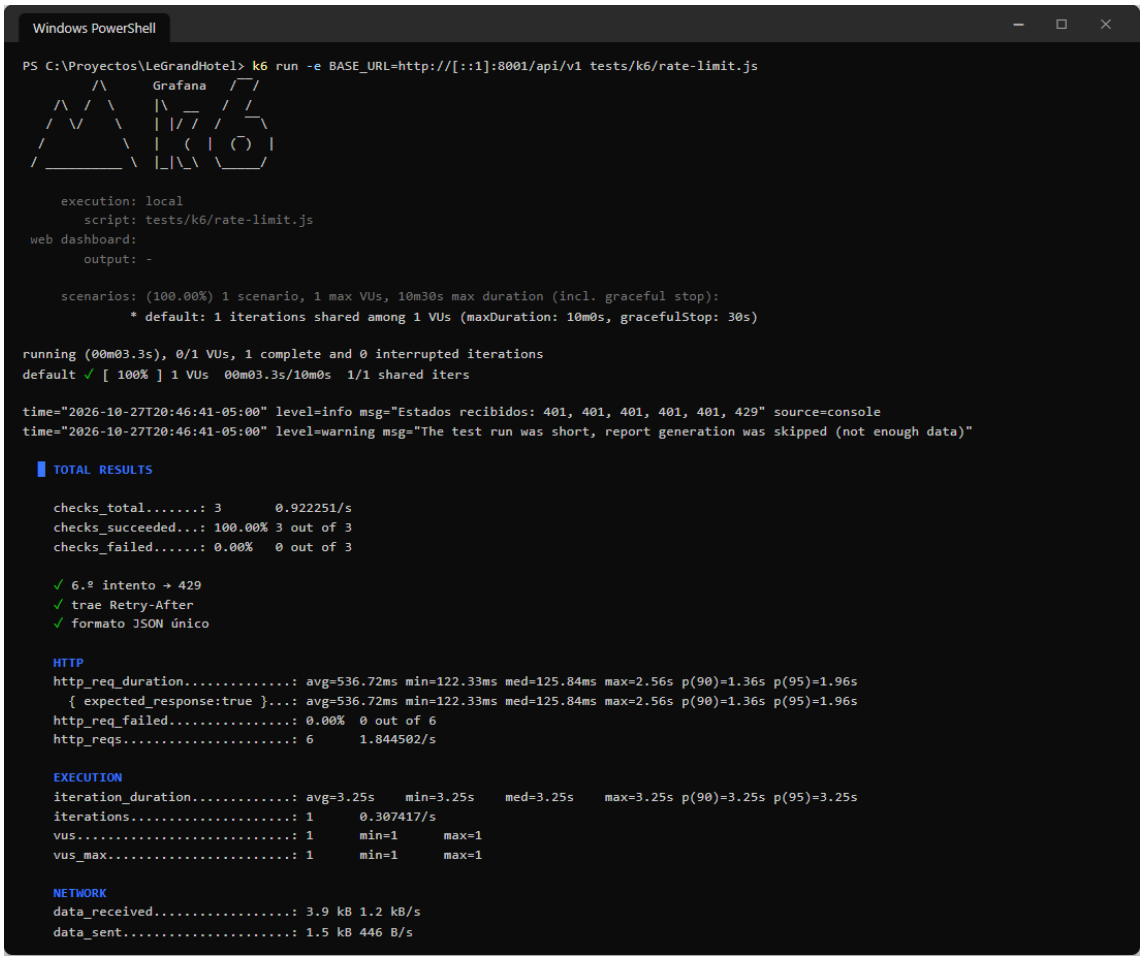


Figura 4. El sexto intento de login recibe 429: la protección funciona.

11. Paso 10: pruebas de carga por escalones

Se corre el mismo guion cuatro veces, cada vez con más usuarios: 5, 10, 25 y 50. Primero se activa el reporte HTML y luego se corre la prueba:

PowerShell

```
$env:K6_WEB_DASHBOARD="true"
$env:K6_WEB_DASHBOARD_EXPORT="docs/pruebas/escalon-5vu.html"
k6 run -e BASE_URL=http://127.0.0.1:8000/api/v1 -e PROFILE=escalon -e VUS=5 -e
USERS=5 tests/k6/reservas.js
```

cmd

```
set "K6_WEB_DASHBOARD=true"
set "K6_WEB_DASHBOARD_EXPORT=docs/pruebas/escalon-5vu.html"
k6 run -e BASE_URL=http://127.0.0.1:8000/api/v1 -e PROFILE=escalon -e VUS=5 -e
USERS=5 tests/k6/reservas.js
```

Luego repite las líneas 2 y 3 cambiando el 5 por 10, después por 25 y por último por 50 (en el nombre del archivo, en VUS y en USERS).

Parte del comando	Qué significa
`K6_WEB_DASHBOARD=true`	Activa el reporte con gráficos
`K6_WEB_DASHBOARD_EXPORT=...`	Dónde guardar el reporte HTML al terminar
`-e PROFILE=escalon`	Sube a los usuarios en 10 s, los mantiene 45 s y baja en 5 s

Parte del comando	Qué significa
`-e VUS=5`	Cuántos usuarios a la vez
`-e USERS=5`	Cuántas cuentas de recepcionista usar (una por usuario virtual)
`tests/k6/reservas.js`	El trabajo de recepción: 70 % consultas, 20 % login, 10 % crear reservas

Por qué: una sola corrida con muchos usuarios dice si la API aguanta, pero no desde cuándo empieza a ponerse lenta. Subir por escalones muestra la tendencia: si el tiempo de respuesta se mantiene, la API escala; si crece con cada escalón, hay un cuello de botella. El escenario imita el trabajo real de la recepción (sobre todo consultas, algunos inicios de sesión y reservas nuevas) para que el resultado se parezca al uso del hotel. El reporte HTML queda como evidencia y muestra cómo cambió todo segundo a segundo.

Qué debes ver: cada corrida dura 1 minuto. Las metas son p95 < 500 ms y menos de 1 % de errores. En la laptop la de velocidad sale con ✗ (ver el paso 11) y la de errores con ✓.



Figura 5. Escalón de 50 usuarios: sin errores, pero el tiempo de respuesta supera la meta.

Para ver el reporte, abre el archivo docs\pruebas\escalon-50vu.html con doble clic:

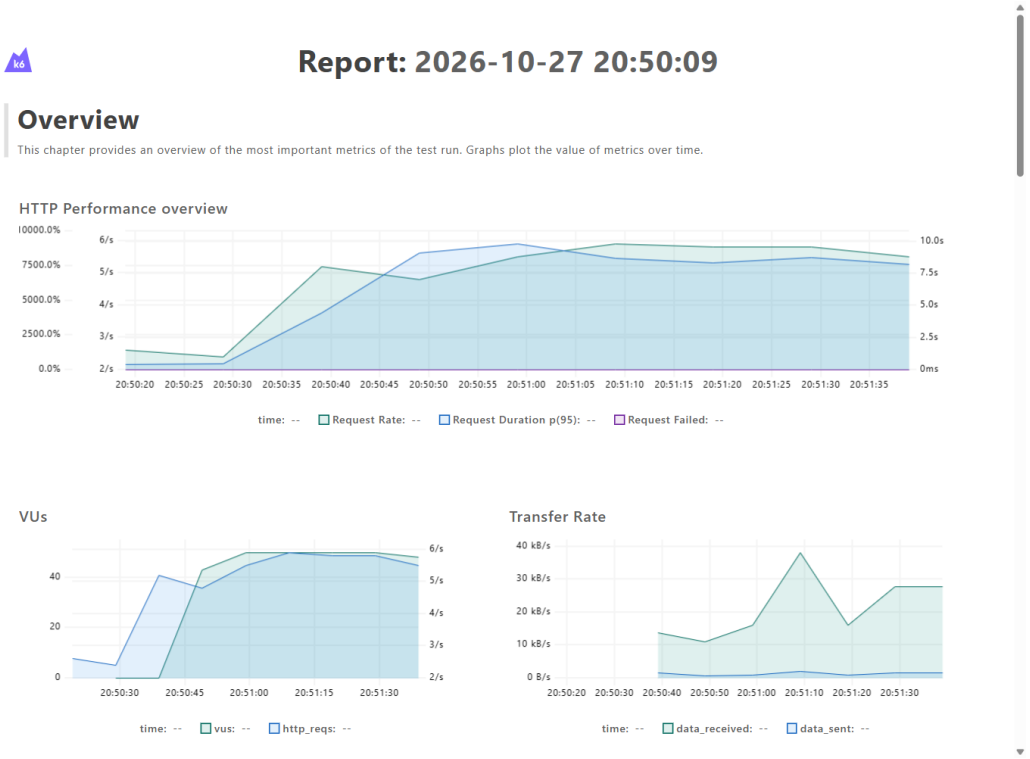


Figura 6. Reporte HTML: usuarios, peticiones por segundo y tiempos durante la prueba.

12. Paso 11: leer los resultados

Al final de cada prueba k6 muestra un resumen. Estos son los datos que hay que mirar, en este orden:

Orden	Dónde mirar	Qué significa	Está bien si…
1	THRESHOLDS	Las metas de la prueba	Todas tienen ✓
2	http_req_failed	Porcentaje de peticiones con error	0 % o casi
3	checks_succeeded	Verificaciones del guion que pasaron	100 %
4	http_req_duration p(95)	El 95 % de las respuestas tardó menos que esto	Menor a la meta (500 ms)
5	http_req_duration med	La mitad de las respuestas tardó menos que esto (lo típico)	Bastante menor que p95
6	http_reqs	Peticiones totales y por segundo	Sube cuando suben los usuarios

Por qué se mira p95 y no el promedio: el promedio esconde a los usuarios que esperan mucho. p95 dice cuánto esperó casi todo el mundo (95 de cada 100), que es lo que sienten las personas en la recepción.

Si una meta sale con ✗: k6 muestra «thresholds ... have been crossed» y termina con código 99. Eso no es un error de k6, es k6 diciendo que la prueba no se aprobó.

12.1 Qué salió en nuestras pruebas

Usuarios	p95	Pet./s	Errores
5	610 ms	4.8	0 %
10	1.8 s	4.9	0 %
25	4.6 s	4.8	0 %
50	9.3 s	4.8	0 %

Las peticiones por segundo se quedan fijas en ~4.8 y el tiempo crece con los usuarios: es la señal de un cuello de botella. La causa es el servidor integrado de PHP, que atiende una petición a la vez. No hubo ningún error: la API va lenta, pero no se cae. En AWS, con Nginx y PHP-FPM (varios procesos a la vez), las mismas pruebas deben cumplir la meta. El detalle está en el «Informe de pruebas de carga con k6».

13. Paso 12: limpiar y dejar todo como estaba

1. Borra las reservas que crearon los usuarios virtuales. Antes de la primera prueba, la última reserva era la 5000 (puedes confirmarlo con el primer comando antes de empezar):

PowerShell
```
cd api
php artisan tinker --execute="echo App\Models\Reservation::max('id');"
php artisan tinker --execute="echo App\Models\Reservation::where('id', '>', 5000)-
>delete();"
```

2. Vuelve a poner los límites normales en api\.env y limpia la configuración:

api\.env
```
RATE_LIMIT_API=60
RATE_LIMIT_LOGIN=5
RATE_LIMIT_REGISTER=3
```

PowerShell
```
php artisan config:clear
```

3. Apaga la API con Ctrl + C en su terminal y, si ya no la usas, la base de datos:

PowerShell
```
docker compose stop mysql
```

Por qué: la prueba crea reservas falsas a nombre de huéspedes de demostración, que ensucian los reportes de ocupación e ingresos. Y con los límites en 100000 el login queda sin protección contra quien prueba contraseñas: hay que devolverlos a sus valores normales. Los usuarios k6-… pueden quedarse para la siguiente prueba; son solo de la base local.

14. Si algo sale mal

Lo que ves	Por qué pasa	Qué hacer
«k6 no se reconoce como un comando»	La terminal se abrió antes de instalar k6	Cierra la terminal y abre una nueva

Lo que ves	Por qué pasa	Qué hacer
100 % de peticiones fallidas (http_req_failed 100 %)	La API escucha en IPv6 y k6 llama por IPv4, o al revés	Usa 127.0.0.1 en el comando de la API y en BASE_URL
setup: «La API no está sana ... 503»	El contenedor nginx de Docker ocupa el puerto 8000	docker compose down y luego docker compose up -d mysql
«No se pudo iniciar sesión con k6-0@... (401)»	Faltan los usuarios de prueba	Repite el paso 4
«No se pudo iniciar sesión ... (429)»	Los límites siguen bajos	Repite el paso 5 (incluido config:clear) y reinicia la API
«No hay clientes: corre MassiveDataSeeder»	La base no tiene huéspedes	Repite el paso 3
La prueba de login da 401 en los 6 intentos	La API del puerto 8001 no tomó el límite de 5	Levántala con la variable RATE_LIMIT_LOGIN=5 en esa misma terminal
En cmd la variable no funciona	set sin comillas guarda espacios de más	Escribe set "VARIABLE=valor"
«report generation was skipped»	La prueba duró muy poco para hacer gráficos	Es normal en la prueba de login; no es un error

15. Resumen de comandos

Todo el recorrido en PowerShell, en orden:

PowerShell

```
# Terminal 1 (preparación y API)
docker compose up -d mysql
cd api
php artisan migrate
php artisan db:seed --class=DemoSeeder
php artisan db:seed --class=MassiveDataSeeder
$env:K6_USERS="50"; php artisan db:seed --class=LoadTestUsersSeeder
# (editar api\.env: RATE_LIMIT_* = 100000)
php artisan config:clear
php -S 127.0.0.1:8000 -t public public/index.php

# Terminal 2 (pruebas, desde la carpeta del proyecto)
curl.exe -s http://127.0.0.1:8000/api/v1/health
k6 run -e BASE_URL=http://127.0.0.1:8000/api/v1 -e VUS=5 -e DURATION=30s
tests/k6/smoke.js
k6 run -e BASE_URL=http://127.0.0.1:8001/api/v1 tests/k6/rate-limit.js   # con la
API 8001 encendida
$env:K6_WEB_DASHBOARD="true"
$env:K6_WEB_DASHBOARD_EXPORT="docs/pruebas/escalon-50vu.html"
k6 run -e BASE_URL=http://127.0.0.1:8000/api/v1 -e PROFILE=escalon -e VUS=50 -e
USERS=50 tests/k6/reservas.js
```